
CprE 458/558: Real-Time Systems

Introduction to Real-Time Systems

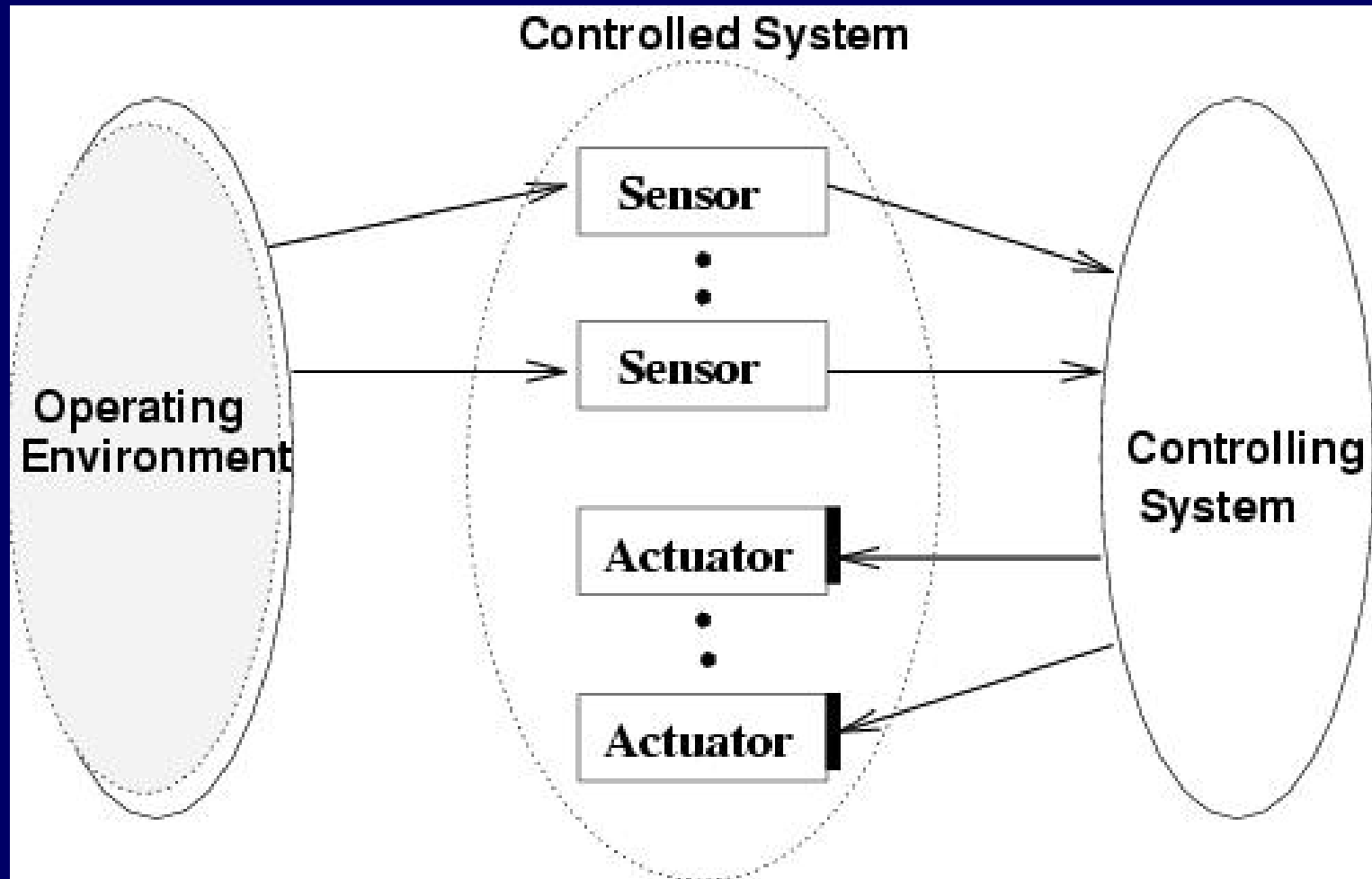
Real-time Systems -- Defined

- Real-time systems are defined as those systems in which the correctness of the system depends not only on the **logical result** of computation, but also on the **time** at which the results are produced.
- The performance of real-time systems must be predictable.
- Real-time systems often operate in a constrained environment – workload variations, fault conditions, resource constraints.

Real-time System – Examples

- Hard real-time systems (e.g., Avionics, Command & Control Systems).
- Firm real-time systems (e.g., radar tracking, manufacturing assembly line).
- Soft real-time systems (e.g., Video conferencing, multimedia applications).

A typical real-time system

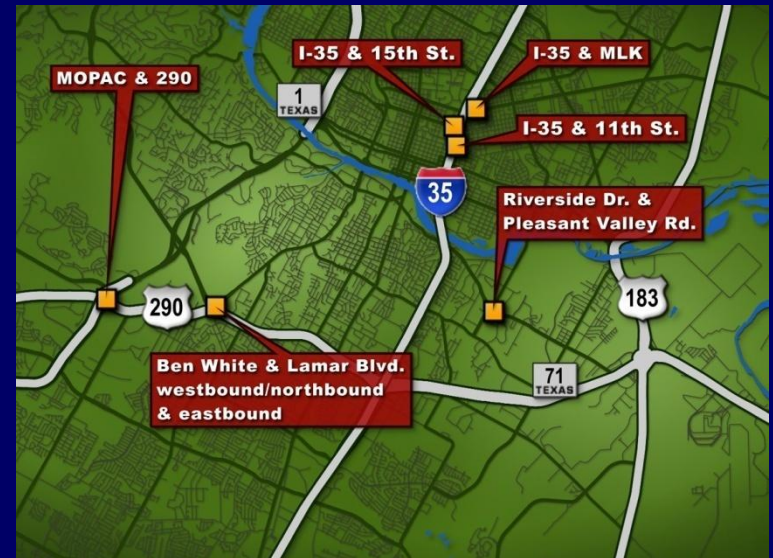


Sample Applications

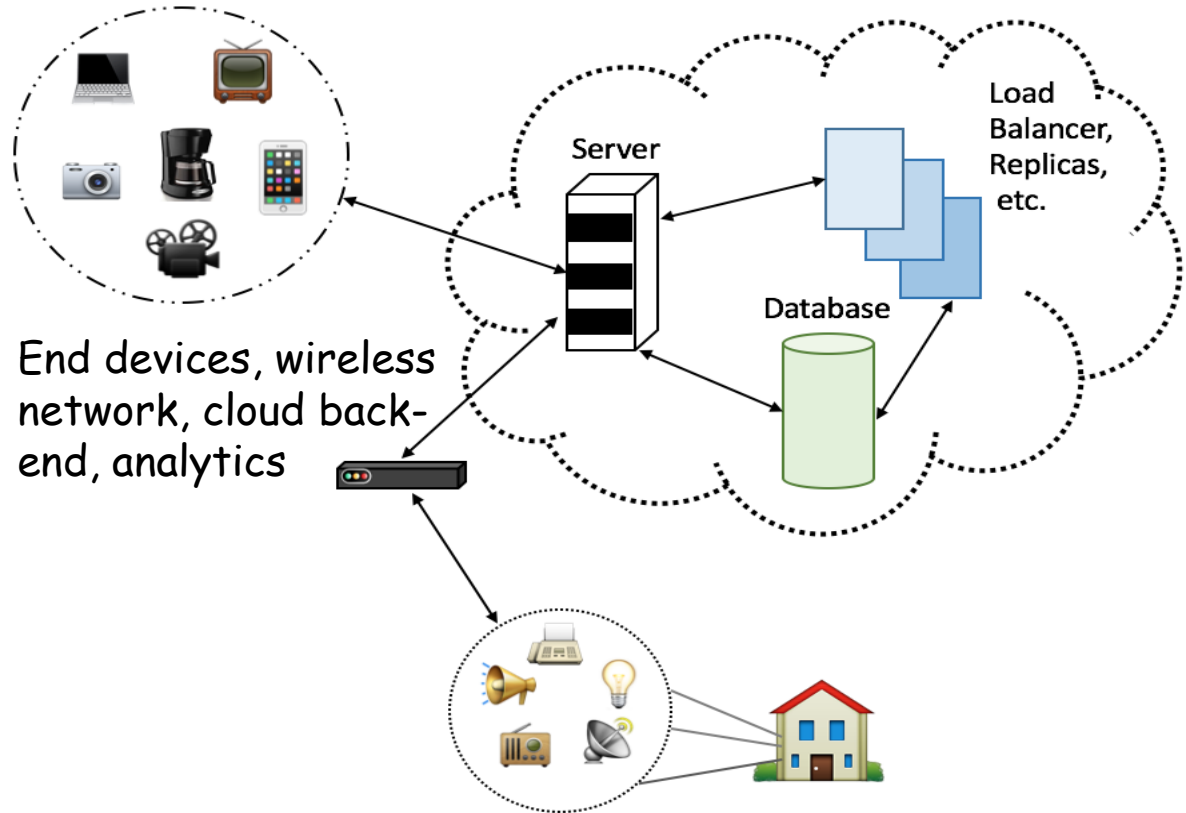
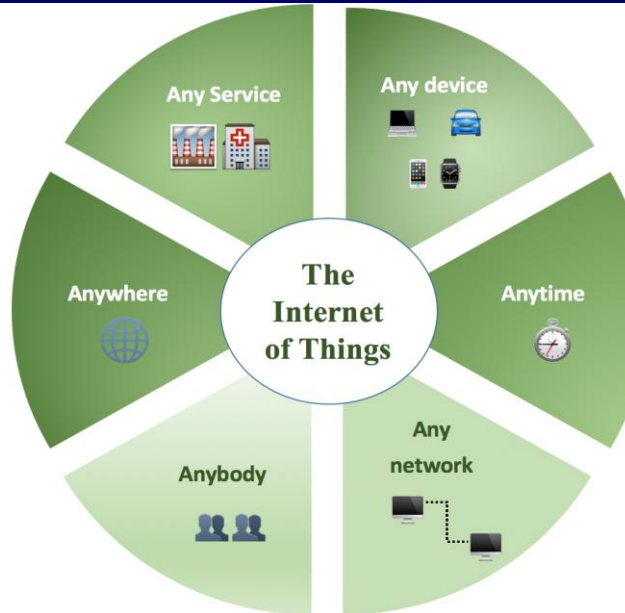
Agile Manufacturing



Traffic control system

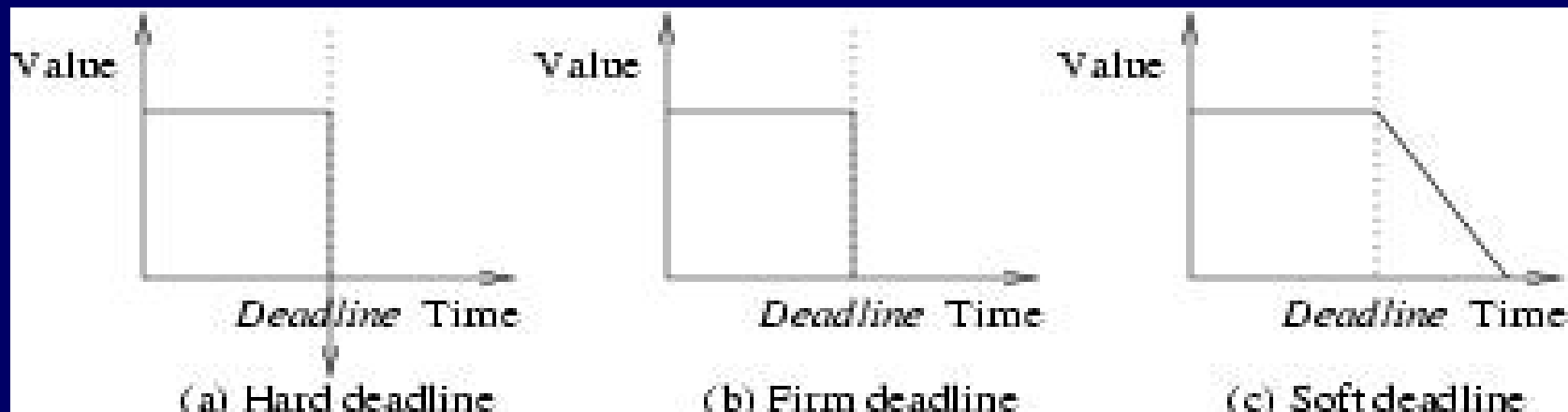


Industrial Internet & Internet of Things



Real-Time Systems -- Introduction

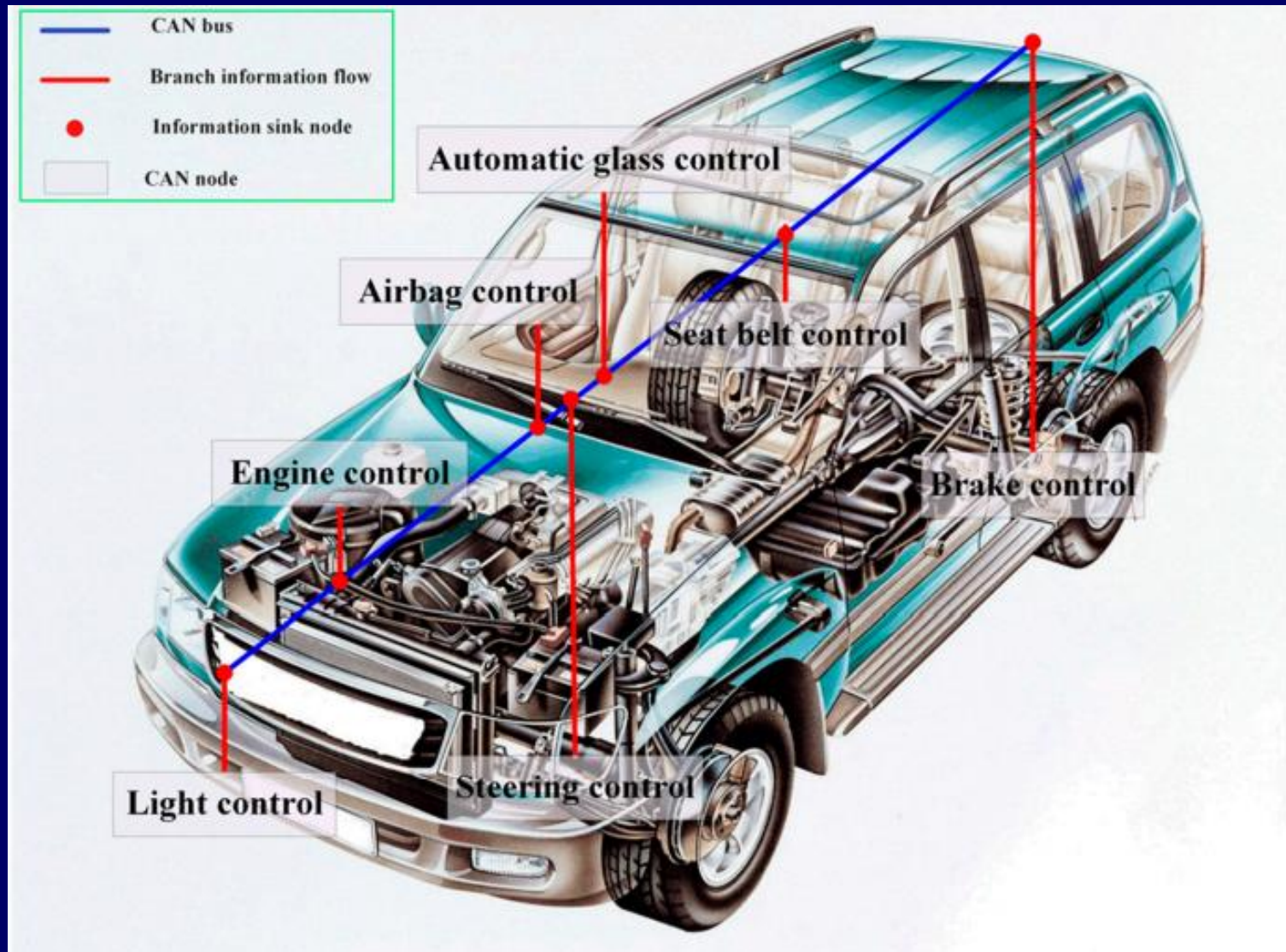
- Hard deadline: penalty due to missing deadline is a higher order of magnitude than the reward in meeting the deadline
- Firm deadline: penalty and reward are in the same order of magnitude
- Soft deadline: penalty often lesser magnitude than reward



Example – Car Driver



Controller area network (CAN) bus in an autonomous vehicle



Example – Car driver

- ***Mission:*** Reaching the destination safely.
- **Controlled System:** Car.
- **Operating environment:** Road conditions.
- **Controlling System**
 - *Human driver:* Sensors - Eyes and Ears of the driver.
 - *Computer:* Sensors - Cameras, Infrared receiver, Laser telemeter, Navigation system, Street maps.
- **Controls:** Accelerator, Steering wheel, Break-pedal.
- **Actuators:** Wheels, Engines, and Brakes.

Example – Car driver (contd)

- **Critical tasks:** Steering and breaking.
- **Non-critical tasks:** Turning on radio.
- **Performance** is not an absolute one. It measures the goodness of the outcome relative to the best outcome possible under a given circumstance.
- **Cost** of fulfilling the mission → Efficient solution.
- **Reliability** of the driver → Fault-tolerance is a must.

Real-Time Tasks (Workload)

- **Periodic tasks**

- Time-driven. Characteristics are known a priori
- Task T_i is characterized by (c_i, p_i)
E.g.: Task monitoring temperature of a patient in an ICU.

- **Aperiodic tasks**

- Event-driven. Characteristics are not known a priori
- Task T_i is characterized by (a_i, r_i, c_i, d_i)
E.g.: Task activated upon detecting change in patient's condition.

- **Sporadic Tasks**

- Known minimum inter-arrival time among successive instances of a (periodic) task, rather strictly being periodic.

p_i : task period a_i : arrival time r_i : ready time
 d_i : deadline c_i : worst-case execution time.

Task constraints

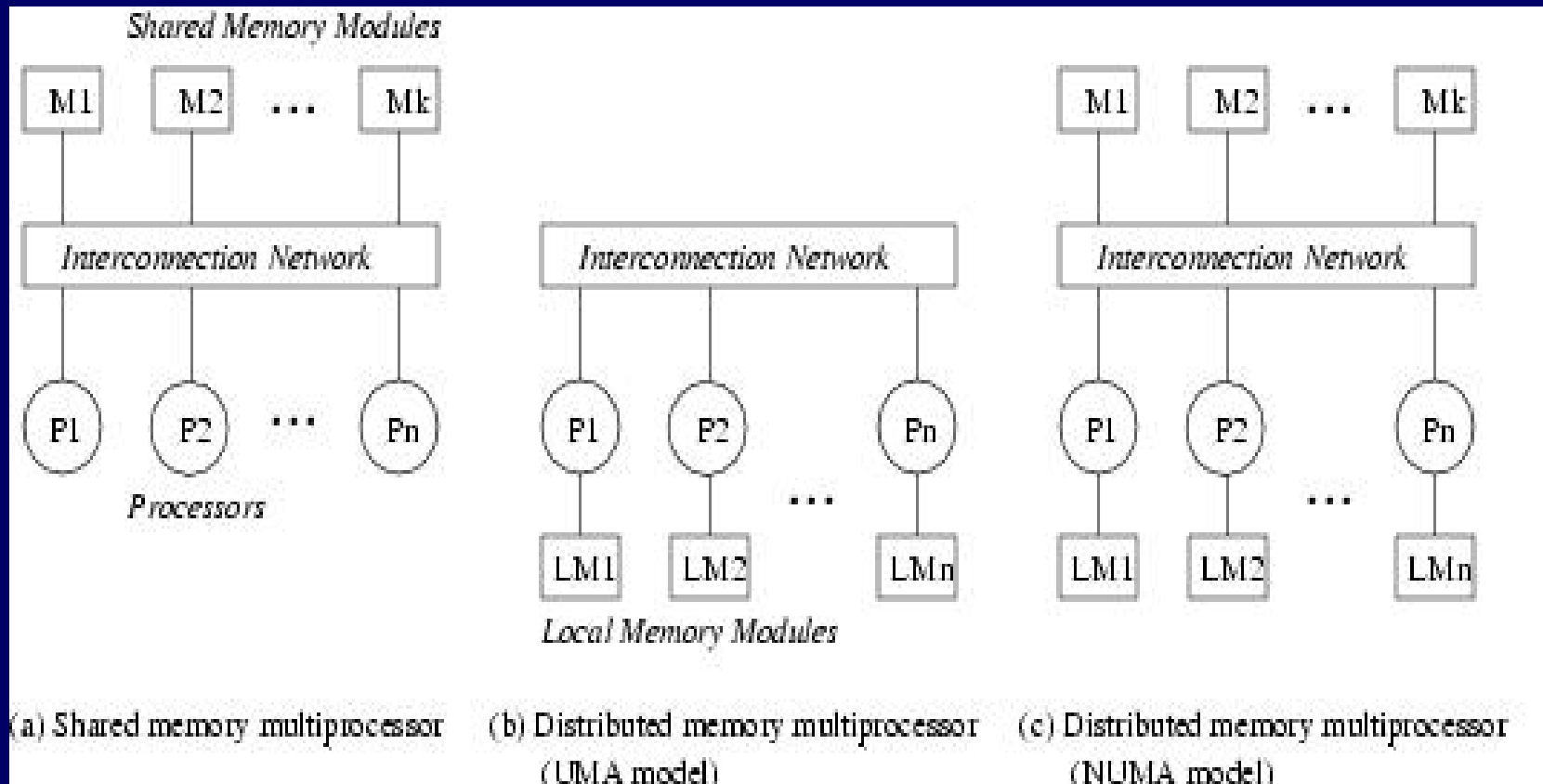
- Deadline constraint
- Resource constraints
 - Shared access (read-read)
 - Exclusive access (write-x, where x: read or write)
- Precedence constraints
 - Task T1 precedes T2 denoted as $T1 \rightarrow T2$
 - i.e., Task T2 can start its execution only after T1 finishes
 - Precedence relations among tasks are denoted in the form of a graph, known as precedence graph
- Fault-tolerant Requirements
 - Redundancy in task execution to achieve higher reliability

Notion of Predictability

- The most common denominator that is expected from a real-time system is *predictability*.
 - The behavior of the real-time system must be predictable which means that with certain assumptions about workload and failures, it should be possible to show at “design time” that all the timing constraints of the application will be met.
- For static systems, 100% guarantees can be given at design time.
- For dynamic systems, 100% guarantee cannot be given since the characteristics of tasks are not known a priori.
- In dynamic systems, predictability means that once a task is admitted into the system (based on online admission test), its guarantee should never be violated as long as the assumptions under which the task was admitted hold.

Computing systems

Uniprocessor, multiprocessor (multicore systems), distributed system, networked control systems



Common Misconceptions

- Real-time computing is equivalent to fast computing.
- Real-time programming is assembly coding, priority interrupt programming, and writing device drivers.
- Real-time systems operate in a static environment.
- The problems in real-time system design have all been solved in other areas of computer science.